# RAPPORT DU PROJET

# ASD

NIVEAU D'ÉTUDE : I ÈRE ANNÉE LICENCE NATIONALE
FILIÈRE : INFORMATIQUE DE GESTION
GROUPE N°:5

RAPPORT PRÉSENTÉ PAR : AMIRA MIDANI E
EYA IBN ABDELHEDI



# THANK YOU WORD :

WE WOULD LIKE TO TAKE A MOMENT TO EXPRESS OUR HEARTFELT THANKS TO OUR MENTORS AND TEACHERS FOR THEIR CONSTANT SUPPORT, VALUABLE GUIDANCE, AND CONSTRUCTIVE FEEDBACK THROUGHOUT THE PROJECT.

HAYET KHMIRI
ZIED EL OUADI
KAOUTHER DRIDI

# “ ANALYSE ET EXPLORATION D’UN RÉSEAU SOCIAL”

# Présentation

## Introduction

Le but de ce projet est de modéliser un réseau social sous forme de graphe pondéré, où chaque utilisateur est un sommet et chaque relation une arête pondérée par la force de la connexion. Ce projet permet de mettre en pratique les structures de données vues en cours, ainsi que des algorithmes fondamentaux tels que DFS, BFS, Dijkstra et la détection de communautés.

## Environnement

Langage de programmation : C
IDE : Code::Blocks
Compilation avec gcc

## Organisation du projet

Chaque fichier source joue un rôle bien défini dans le fonctionnement global du projet. Voici une description individuelle de chacun.
Le projet est composé de plusieurs fichiers pour assurer une bonne structure modulaire .

# Bibliothèques utilisées

## #include <stdio.h>

La bibliothèque standard d'entrée/sortie permet de réaliser des opérations de lecture et d'écriture, telles que l'affichage sur la console (printf) et la lecture de données à partir de l'utilisateur (scanf)

## #include <stdlib.h>

Elle sert à gérer la mémoire dynamique (avec malloc, calloc, realloc, free) et à fournir des fonctions utiles pour convertir des types ou quitter le programme (exit). C'est essentiel pour créer et gérer ton graphe et ses structures de données de manière flexible.

```
#include <stdio.h>
#include <stdlib.h>
```

# fichier source

## ELTUSER.h

Ce fichier contient la déclaration de la structure ELEMENT représentant un utilisateur, ainsi que les prototypes des fonctions associées à la gestion d'un utilisateur (création, affichage, comparaison, copie, destruction).

```c
#ifndef ELTUSER_H_INCLUDED
#define ELTUSER_H_INCLUDED
typedef struct {
    int id;
    char nom[50];
    int age;
} UtilisateurStruct, *ELEMENT;

ELEMENT elementCreer();
void elementLire(ELEMENT *e);
void elementAfficher(ELEMENT e);
void elementDetruire(ELEMENT e);


#endif // ELTUSER_H_INCLUDED
```

# fichier source

## GRAPHE.h

Contient les structures de données principales du graphe (nœud de la liste d'adjacence et graphe pondéré), ainsi que les prototypes de toutes les fonctions de manipulation du graphe.

```c
#ifndef GRAPHE_H_INCLUDED
#define GRAPHE_H_INCLUDED
#include "ELTUSER.h"

// ==== STRUCTURE DE NOEUD DE LA LISTE D'ADJACENCE ====
typedef struct structNode {
    ELEMENT user;
    int weight;
    struct structNode* next;
} structNode, *Node;

// ==== STRUCTURE DU GRAPHE ====
typedef struct {
    int V;
    ELEMENT* utilisateurs;
    Node* adjList;
} *graphePondere;


//les primitives
graphePondere creerGraphe(int V);
void detruireGraphe(graphePondere g);
int ajouterUtilisateur(graphePondere g, ELEMENT user);
int ajouterRelation(graphePondere g, int src, int dest, int poids);
int supprimerRelation(graphePondere g, int src, int dest);
int supprimerUtilisateur(graphePondere g, int idUser);
void afficherGraphe(graphePondere g);
graphePondere chargerGraphe(const char* filename);
void sauvegarderGraphe(graphePondere g, const char* filename);

//les algorithmes
void dfs(graphePondere g, int idUser);
void bfs(graphePondere g, int idUser);
void dijkstra(graphePondere g, int src_id, int dest_id);
void trouver_communautes(graphePondere g);
#endif
```

# ELTUSER.c

## 1. elementCreer :

```
ELEMENT elementCreer() {
    ELEMENT e = (ELEMENT)malloc(sizeof(UtilisateurStruct));
    if (e) {
        e->id = 0;
        strcpy(e->nom, "");
        e->age = 0;
    }
    return e;
}
```

Crée dynamiquement un utilisateur avec ID, nom et âge, et retourne un pointeur vers celui-ci.

## 2. elementLire :

```
void elementLire(ELEMENT *e) {
    *e = elementCreer();
    printf("ID : "); scanf("%d", &(*e)->id);
    getchar();
    printf("Nom : "); fgets((*e)->nom, 50, stdin);
    (*e)->nom[strcspn((*e)->nom, "\n")] = '\0';
    printf("Age : "); scanf("%d", &(*e)->age);
}
```

Crée dynamiquement un utilisateur avec ID, nom et âge, et retourne un pointeur vers celui-ci.

## 3.elementAfficher

```
void elementAfficher(ELEMENT e) {
    printf("ID: %d | Nom: %s | Âge: %d\n", e->id, e->nom, e->age);
}
```

## 4.elementDetruire:

```
void elementDetruire(ELEMENT e) {
    free(e);
}
```

Libère la mémoire allouée à un utilisateur

# GRAPHE.c

## 1.creerGraphe(int V) :

```
graphePondere creerGraphe(int V) {
    graphePondere g = (graphePondere)malloc(sizeof(*g));
    if (!g) return NULL;

    g->V = V;
    g->utilisateurs = (ELEMENT*)malloc(V * sizeof(ELEMENT));
    g->adjList = (Node*)malloc(V * sizeof(Node));

    for (int i = 0; i < V; i++) {
        g->utilisateurs[i] = NULL;
        g->adjList[i] = NULL;
    }

    return g;
}
```

Initialise la structure du graphe (utilisateurs + listes d'adjacence) avec un nombre donné de sommets.

## 2.detruireGraphe(graphePondere g):

```
void detruireGraphe(graphePondere g) {
    if (!g) return;

    for (int i = 0; i < g->V; i++) {
        elementDetruire(g->utilisateurs[i]);

        Node curr = g->adjList[i];
        while (curr) {
            Node tmp = curr;
            curr = curr->next;
            free(tmp);
        }
    }

    free(g->utilisateurs);
    free(g->adjList);
    free(g);
}
```

Libère toute la mémoire utilisée par le graphe (utilisateurs et relations).

# GRAPHE.c

## 3.int ajouterUtilisateur(graphePondere g, ELEMENT user)

```
int ajouterUtilisateur(graphePondere g, ELEMENT user) {
    if (!g || !user || user->id < 0)
        return 0;

    // Redimensionner si id > V - 1
    if (user->id >= g->V) {
        int newSize = user->id + 1;

        ELEMENT* newUsers = realloc(g->utilisateurs, newSize * sizeof(ELEMENT));
        Node* newAdj = realloc(g->adjList, newSize * sizeof(Node));

        if (!newUsers || !newAdj) {
            printf("Erreur de reallocation memoire.\n");
            return 0;
        }

        // Initialiser les nouvelles cases
        for (int i = g->V; i < newSize; i++) {
            newUsers[i] = NULL;
            newAdj[i] = NULL;
        }

        g->utilisateurs = newUsers;
        g->adjList = newAdj;
        g->V = newSize; // Mise à jour de la taille du graphe
    }

    if (g->utilisateurs[user->id] != NULL)
        return 0; // utilisateur déjà existant

    ELEMENT copie = elementCreer();
    copie->id = user->id;
    copie->age = user->age;
    strcpy(copie->nom, user->nom);

    g->utilisateurs[user->id] = copie;
    return 1;
}
```

Cette fonction ajoute un nouvel utilisateur au graphe s'il n'existe pas déjà. Elle insère le nouvel élément dans la liste des sommets du graphe en respectant la structure et la taille maximale.

# GRAPHE.c

## 4.void ajouterRelation(graphePondere g, int src, int dest, int poids)

```
int ajouterRelation(graphePondere g, int src, int dest, int poids) {
    if (!g || src < 0 || dest < 0 || src >= g->V || dest >= g->V)
        return 0;

    if (!g->utilisateurs[src] || !g->utilisateurs[dest])
        return 0;

    // Vérifier si la relation existe déjà (dans src -> dest)
    Node current = g->adjList[src];
    while (current) {
        if (current->user->id == dest) {
            current->weight = poids; // mise à jour du poids
            return 1;
        }
        current = current->next;
    }

    // Ajouter la nouvelle relation dans src -> dest
    Node newNode = (Node)malloc(sizeof(structNode));
    newNode->user = g->utilisateurs[dest];
    newNode->weight = poids;
    newNode->next = g->adjList[src];
    g->adjList[src] = newNode;

    // Si relation bidirectionnelle : ajouter dest -> src
    Node newNode2 = (Node)malloc(sizeof(structNode));
    newNode2->user = g->utilisateurs[src];
    newNode2->weight = poids;
    newNode2->next = g->adjList[dest];
    g->adjList[dest] = newNode2;

    return 1;
```

Cette fonction crée une relation pondérée entre deux utilisateurs existants du graphe. Elle ajoute une arête dans la liste d'adjacence du sommet source vers le sommet destination avec un poids donné, tout en vérifiant l'existence des deux utilisateurs.

# GRAPHE.c

## 5.void supprimerRelation(graphePondere g, int src, int dest)

```c
int supprimerRelation(graphePondere g, int src, int dest) {
    if (!g || src < 0 || dest < 0 || src >= g->V || dest >= g->V)
        return 0;

    // Supprimer dans src -> dest
    Node *p = &g->adjList[src];
    while (*p) {
        if ((*p)->user->id == dest) {
            Node tmp = *p;
            *p = (*p)->next;
            free(tmp);
            break;
        }
        p = &(*p)->next;
    }


    // Supprimer dans dest -> src (relation bidirectionnelle)
    p = &g->adjList[dest];
    while (*p) {
        if ((*p)->user->id == src) {
            Node tmp = *p;
            *p = (*p)->next;
            free(tmp);
            break;
        }
        p = &(*p)->next;
    }

    return 1;
}
```

# GRAPHE.c

- Supprime la relation entre deux utilisateurs (dans les deux sens), sans supprimer les utilisateurs eux-mêmes.

## 6. void supprimerUtilisateur(graphePondere g, int idUser):

```c
int supprimerUtilisateur(graphePondere g, int idUser) {
    if (!g || idUser < 0 || idUser >= g->V || !g->utilisateurs[idUser])
        return 0;

    // Supprimer toutes les relations vers cet utilisateur
    for (int i = 0; i < g->V; i++) {
        if (i != idUser) {
            supprimerRelation(g, i, idUser);
        }
    }

    // Supprimer ses propres relations
    Node curr = g->adjList[idUser];
    while (curr) {
        Node tmp = curr;
        curr = curr->next;
        free(tmp);
    }
    g->adjList[idUser] = NULL;
```

# GRAPHE.c

```
    // Supprimer l'utilisateur
    elementDetruire(g->utilisateurs[idUser]);
    g->utilisateurs[idUser] = NULL;

    return 1;
}
```

- Supprime un utilisateur donné et toutes les relations où il apparaît. Libère la mémoire liée à ses relations.

## 7. void afficherGraphe(graphePondere g) :

```
void afficherGraphe(graphePondere g) {
    if (!g) return;

    printf("=== Graphe du Réseau Social ===\n");
    for (int i = 0; i < g->V; i++) {
        if (g->utilisateurs[i]) {
            printf("%s (%d)", g->utilisateurs[i]->nom, g->utilisateurs[i]->id);

            Node curr = g->adjList[i];
            while (curr) {
                printf(" --%d--> %s (%d)", curr->weight, curr->user->nom, curr->user->id);
                curr = curr->next;
            }
            printf("\n");
        }
    }
}
```

Affiche tous les utilisateurs du graphe, ainsi que leurs relations (amis) et les poids associés.

# GRAPHE.c

## 8. graphePondere chargerGraphe(const char* nomFichier):

```
graphePondere chargerGraphe(const char* filename) {
    FILE* f = fopen(filename, "r");
    if (!f) {
        printf("Erreur : fichier %s introuvable.\n", filename);
        return NULL;
    }

    // Étape 1 : Compter le nombre d'utilisateurs pour connaître V
    int maxId = -1;
    char ligne[100];
    while (fgets(ligne, sizeof(ligne), f)) {
        if (ligne char chargerGraphe::ligne
            int id, age;
            char nom[50];
            if (sscanf(ligne, "U %d %s %d", &id, nom, &age) == 3) {
                if (id > maxId) maxId = id;
            }
        }
    }

// Création du graphe avec V = maxId + 1
graphePondere g = creerGraphe(maxId + 1);
rewind(f); // retour au début du fichier

// Étape 2 : Lecture réelle du fichier
while (fgets(ligne, sizeof(ligne), f)) {
    if (ligne[0] == 'U') {
        int id, age;
        char nom[50];
        if (sscanf(ligne, "U %d %s %d", &id, nom, &age) == 3) {
            ELEMENT e = elementCreer();
            e->id = id;
            strcpy(e->nom, nom);
            e->age = age;
            ajouterUtilisateur(g, e);
            elementDetruire(e); // détruire la copie locale
        }
    } else if (ligne[0] == 'R') {
        int src, dest, poids;
        if (sscanf(ligne, "R %d %d %d", &src, &dest, &poids) == 3) {
            ajouterRelation(g, src, dest, poids);
        }
    }
}
```

```
// Supprimer l'utilisateur
elementDetruire(g->utilisateurs[idUser]);
g->utilisateurs[idUser] = NULL;

return 1;
```

Lit un fichier texte formaté contenant les utilisateurs et leurs relations. Reconstruit le graphe en mémoire.

## 9. void sauvegarderGraphe(graphePondere g, const char* nomFichier):

```
void sauvegarderGraphe(graphePondere g, const char* filename) {
    FILE* f = fopen(filename, "w");
    if (!f) {
        printf("Erreur : impossible d'ouvrir le fichier %s pour l'écriture.\n", filename);
        return;
    }

    // Écriture des utilisateurs
    for (int i = 0; i < g->V; i++) {
        if (g->utilisateurs[i]) {
            fprintf(f, "U %d %s %d\n", g->utilisateurs[i]->id, g->utilisateurs[i]->nom, g->utilisateurs[i]->age);
        }
    }

    // Écriture des relations
    for (int i = 0; i < g->V; i++) {
        Node curr = g->adjList[i];
        while (curr) {
            int id1 = i;
            int id2 = curr->user->id;

            // Pour éviter les doublons, on n'écrit qu'une seule direction (id1 < id2)
            if (id1 < id2) {
                fprintf(f, "R %d %d %d\n", id1, id2, curr->weight);
            }

            curr = curr->next;
        }
    }

    fclose(f);
    printf("Graphe sauvegardé dans le fichier %s.\n", filename);
}
```

Sauvegarde le graphe actuel dans un fichier texte (utilisateurs, puis relations), pour le recharger plus tard.

# Algorithmes d'analyse utilisés

## 1. DFS (Depth-First Search) – Parcours en profondeur

```c
void dfs(graphePondere g, int idUser) {
    int visite[g->V];
    for (int i = 0; i < g->V; i++) visite[i] = 0;

    printf("DFS à partir de %s (%d) :\n", g->utilisateurs[idUser]->nom, idUser);
    dfsRec(g, idUser, visite);
}
void dfsRec(graphePondere g, int id, int visite[]) {
    visite[id] = 1;
    printf("%s (%d)  ", g->utilisateurs[id]->nom, id);

    Node voisin = g->adjList[id];
    while (voisin != NULL) {
        int v = voisin->user->id;
        if (!visite[v]) {
            dfsRec(g, v, visite);
        }
        voisin = voisin->next;
    }
}
```

But : Explorer toutes les connexions d'un utilisateur (noeud) en allant aussi loin que possible dans chaque direction avant de revenir en arrière.

Utilité dans le projet : Identifier les groupes fortement connectés ou vérifier si un chemin existe entre deux utilisateurs.

# Algorithmes d'analyse utilisés

## 2. BFS (Breadth-First Search) – Parcours en largeur

```
void bfs(graphePondere g, int idUser) {

    // Vérifie que l'id est valide et que l'utilisateur existe
    if (idUser < 0 || idUser >= g->V || g->utilisateurs[idUser] == NULL)
        return;

    int* visite = (int*)calloc(g->V, sizeof(int));        // Tableau pour marquer les utilisateurs déjà visités
    int* file = (int*)malloc(g->V * sizeof(int));         // File (queue) pour stocker les utilisateurs à visiter
    int debut = 0, fin = 0;                               // Indices de début et fin de la file

    visite[idUser] = 1;                                   // Marque le point de départ comme visité
    file[fin++] = idUser;                                 // Ajoute l'utilisateur de départ dans la file

    printf("Utilisateurs accessibles depuis %s (BFS) :\n", g->utilisateurs[idUser]->nom);

    while (debut < fin) {                                 // Tant que la file n'est pas vide
        int courant = file[debut++];                      // Prend l'utilisateur au début de la file
        elementAfficher(g->utilisateurs[courant]);        // Affiche ses informations

        Node voisin = g->adjList[courant];                // Récupère la liste de ses voisins

        while (voisin != NULL) {                          // Parcourt tous ses voisins
            int idVoisin = voisin->user->id;              // Récupère l'id du voisin

            if (!visite[idVoisin]) {                      // Si ce voisin n'a pas encore été visité
                visite[idVoisin] = 1;                     // On le marque comme visité
                file[fin++] = idVoisin;                   // Et on l'ajoute à la file pour le traiter plus tard
            }

            voisin = voisin->next;                        // Passe au voisin suivant
        }
    }

    free(visite);                                         // Libère la mémoire du tableau de visite
    free(file);                                           // Libère la mémoire de la file
}
```

- But : Explorer les utilisateurs par niveau de proximité, en partant d'un utilisateur donné.
- Utilité : Trouver le plus court chemin non pondéré entre deux utilisateurs (ex : distance sociale minimale).

# Algorithmes d'analyse utilisés

## 3. Dijkstra

- But : Trouver le plus court chemin pondéré entre un utilisateur source et tous les autres.
- Utilité : Calculer la distance minimale en fonction du poids des relations (par exemple, plus une relation est forte, plus le chemin est "rapide").

```
oid dijkstra(graphePondere g, int src_id, int dest_id) {
    int* distance = (int*)malloc(g->V * sizeof(int));       // distance[i] : distance min connue de src à i
    int* visite = (int*)calloc(g->V, sizeof(int));          // visite[i] : 1 si le sommet i a été traité
    int* precedent = (int*)malloc(g->V * sizeof(int));      // precedent[i] : sommet précédent sur le chemin vers i

    // Initialisation
    for (int i = 0; i < g->V; i++) {
        distance[i] = INF;          // On suppose que tout est infiniment loin au départ
        precedent[i] = -1;          // Pas encore de chemin trouvé
    }

    distance[src_id] = 0;           // Distance de départ à lui-même = 0

    //  Étape principale : répéter pour tous les sommets
    for (int count = 0; count < g->V - 1; count++) {
        int min = INF;
        int u = -1;

        // Trouver le sommet non visité avec la distance minimale
        for (int v = 0; v < g->V; v++) {
            if (!visite[v] && distance[v] < min) {
                min = distance[v];
                u = v;
            }
        }

        // Si aucun sommet non visité accessible → on continue à itérer, mais on ne fait rien ce tour
        if (u != -1) {
            visite[u] = 1;  // Marquer comme visité

            // Mettre à jour les distances de ses voisins
            Node voisin = g->adjList[u];
            while (voisin != NULL) {
                int v = voisin->user->id;

                if (!visite[v] && distance[u] + voisin->weight < distance[v]) {
                    distance[v] = distance[u] + voisin->weight;
                    precedent[v] = u;   // On enregistre le chemin
                }
```

# Algorithmes d'analyse utilisés

## 3. Dijkstra

```
                distance[v] = distance[u] + voisin->weight;
                precedent[v] = u;  // On enregistre le chemin
            }

            voisin = voisin->next;
        }
    }
}

// Affichage du chemin trouvé
if (distance[dest_id] == INF) {
    printf("Aucun chemin entre %s et %s\n", g->utilisateurs[src_id]->nom, g->utilisateurs[dest_id]->nom);
} else {
    printf("Chemin le plus court de %s à %s :\n", g->utilisateurs[src_id]->nom, g->utilisateurs[dest_id]->nom);

    // Reconstruction du chemin (inversé)
    int chemin[100], index = 0;
    int v = dest_id;
    while (v != -1) {
        chemin[index++] = v;
        v = precedent[v];
    }

    // Affichage dans le bon ordre
    for (int i = index - 1; i >= 0; i--) {
        printf("%s", g->utilisateurs[chemin[i]]->nom);
        if (i > 0) printf(" → ");
    }
    printf("\nPoids total : %d\n", distance[dest_id]);
}

// libération mémoire
free(distance);
free(visite);
free(precedent);
```

# Algorithmes d'analyse utilisés

## 4. Détection de communautés

But : Identifier des groupes d'utilisateurs fortement liés entre eux et faiblement connectés aux autres.

Exemples de techniques possibles :

Basée sur DFS/BFS pour regrouper les composants connexes.

Basée sur des heuristiques simples (comme supprimer les arêtes faibles).

```
void trouver_communautes(graphePondere g) {

    int* visite = (int*)calloc(g->V, sizeof(int));      // Marque les utilisateurs déjà visités (0 = non visité)
    int* queue = (int*)malloc(g->V * sizeof(int));      // File pour parcourir chaque communauté avec BFS
    int num = 1;                                        // Numéro de communauté à afficher

    // Parcourir tous les utilisateurs du graphe
    for (int i = 0; i < g->V; i++) {
        // Si l'utilisateur existe et n'a pas encore été visité
        if (g->utilisateurs[i] != NULL && !visite[i]) {
            printf("Communauté %d : ", num++);          // On commence une nouvelle communauté

            int front = 0, rear = 0;                    // Réinitialise les indices de la file
            queue[rear++] = i;                          // Ajoute le premier utilisateur de la communauté
            visite[i] = 1;                               // Le marque comme visité

            // Parcours BFS de la communauté
            while (front < rear) {
                int courant = queue[front++];           // Prend l'utilisateur courant dans la file
                printf("%s ", g->utilisateurs[courant]->nom); // Affiche son nom

                Node voisin = g->adjList[courant];      // Récupère ses voisins

                // Parcourt tous les voisins de l'utilisateur courant
                while (voisin != NULL) {
                    int idVoisin = voisin->user->id;

                    if (!visite[idVoisin]) {            // Si ce voisin n'a pas été visité
                        visite[idVoisin] = 1;           // On le marque
                        queue[rear++] = idVoisin;       // Et on l'ajoute dans la file
                    }

                    voisin = voisin->next;              // Passe au voisin suivant
                }
            }

            printf("\n");                               // Saute une ligne pour passer à la prochaine communauté
        }
    }

    free(visite);                                       // Libère le tableau des visites
    free(queue);                                        // Libère la file
```

# le menu et les choix

## menu principal

```
void afficherMenu() {
    printf("====== MENU RESEAU SOCIAL ======\n");
    printf("1. =====Charger un graphe depuis un fichier=====\n");
    printf("2. =====Ajouter un utilisateur=====\n");
    printf("3. ======Ajouter une relation======\n");
    printf("4. ======Supprimer une relation======\n");
    printf("5. =====Supprimer un utilisateur======\n");
    printf("6. =====Afficher le graphe=====\n");
    printf("7. =====Sauvegarder le graphe dans un fichier=====\n");
    printf("8. =====DFS - Explorer les amis directs et indirects======\n");
    printf("9. ====BFS - Explorer les niveaux d'amis=====\n");
    printf("10.===== Dijkstra - Chemin le plus fort entre deux utilisateurs======\
    printf("11. =====Détecter les communautés=====\n");
    printf("0. =====Quitter=====\n");
    printf("Votre choix : ");
}
```

Lisibilité améliorée par des séparateurs visuels dans le menu.

- La fonction main gère le menu principal du programme, permettant à l'utilisateur de choisir parmi différentes options pour interagir avec un graphe représentant un réseau social. Elle exécute les actions appropriées (ajouter, supprimer, explorer, sauvegarder) en fonction du choix de l'utilisateur et répète cette opération jusqu'à ce que l'utilisateur choisisse de quitter. Avant de terminer, elle libère les ressources allouées pour le graphe.

# graph1.txt.txt

graph1.txt.txt

Fichier Modifier Affichage

```
U 0 Amira 24
U 1 Yassine 27
U 2 Leila 22
U 3 Mehdi 29
U 4 Nour 26
U 5 Sofien 31

R 0 1 4  # Amira - Yassine
R 0 2 5  # Amira - Leila (relation forte)
R 1 3 2  # Yassine - Mehdi
R 2 4 3  # Leila - Nour
R 3 5 4  # Mehdi - Sofien
R 4 5 1  # Nour - Sofien (relation faible)
```

Le fichier graph1.txt contient les données d'un réseau social avec 6 utilisateurs et leurs âges, ainsi que 6 relations pondérées représentant l'intensité des liens (forts ou faibles). Il sert à initialiser le graphe dans le programme pour permettre l'analyse et l'exploration du réseau.

# CLOSING WORD:

IN CONCLUSION, THIS PROJECT HAS BEEN AN INVALUABLE LEARNING EXPERIENCE, ALLOWING US TO GROW BOTH INDIVIDUALLY AND AS A TEAM. IT CHALLENGED US TO THINK CRITICALLY, COLLABORATE EFFECTIVELY, AND ADAPT TO NEW PROGRAMMING TECHNIQUES. WE ARE PROUD OF THE SOLUTION WE HAVE DEVELOPED AND CONFIDENT THAT IT DEMONSTRATES OUR ABILITY TO OVERCOME CHALLENGES AND INNOVATE.

we hope you enjoyed it